



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2133 – Estructuras de Datos y Algoritmos

2025 - 2

Tarea 2

Fecha de entrega código: 24 de octubre, 23:59 Chile continental.

Link a generador de repos: [CLICK AQUÍ](#)

Objetivos

- Emplear optimizaciones de búsqueda usando hashing.
- Diseñar una estrategia de backtracking para solucionar un problema de restricciones.

Introducción



Has sido contactado tanto por Gru como por su enemigo Vector. Gru ha desarrollado un lenguaje para comunicarse con los minions y, con tu ayuda, espera crear un diccionario que detalle todas las nuevas palabras. Por otro lado, Vector requiere de tu ayuda para descifrar los mensajes que envía Gru a los minions. Lo dudas un momento, pero decides no elegir bandos y ayudarlo a él también. Además, debes estar atento, porque Gru está trabajando en construir su arma definitiva, por lo que podría acudir a ti para culminar su gran trabajo.

Parte 1: ¡Creando el Diccionario Banana!



Problema

Gru ha creado su propio lenguaje para comunicarse con los minions sin filtrar información al enemigo. Para facilitar su uso, decide confeccionar el diccionario **banana**. Con el fin de llevar a cabo su plan, Gru solicita tu ayuda como experto en estructuras de datos. Tu objetivo es construir un diccionario que permita consultas eficientes. Constantemente se reciben palabras nuevas, consultas por palabras exactas, palabras que empiezan con un prefijo dado, y palabras que terminan con un sufijo dado.

Para cumplir tu misión, primero recibirás un entero E , que indica la cantidad total de eventos. Luego, deberás resolver los siguientes eventos:

2.1 - Eventos

WRITE {W} {words}

Este evento indica que se escribirán W palabras nuevas en el diccionario.

input	
1	WRITE {amount}
2	palabra_1
3	.
4	.
5	.
6	palabra_W

Se debe mostrar el siguiente output:

output	
1	Se han escrito {W} palabras nuevas en el diccionario.

Importante: La complejidad de este evento no debe ser mayor a $O(L)$ por cada palabra escrita, donde L es el largo de la palabra.

FIND-WORD {word}

Recibes una palabra y debes indicar si se encuentra o no en el diccionario.

input	
1	FIND-WORD {word}

Debes entregar el siguiente output:

output	
1	Palabra {word} encontrada.

y en el caso de que la palabra no esté escrita en el diccionario:

output	
1	Palabra {word} no encontrada.

Importante: Se requiere que esta operación tenga una complejidad no mayor a $O(1)$ en caso promedio.

FIND-PREFIX {prefix}

En este evento recibirás un string **prefix** y deberás entregar todas las palabras del diccionario que tienen ese prefijo, es decir, que comienzan con el string **prefix**.

Se debe mostrar el siguiente output:

output	
1	Prefijo {prefix}:
2	palabra_1
3	.
4	.
5	.
6	palabra_k

Importante: La operación debe tener complejidad no mayor a $O(k \times L)$, donde k es la cantidad de palabras encontradas y L el largo de la palabra más larga encontrada.

FIND-SUFFIX {suffix}

En este evento recibirás un string **suffix** y deberás entregar todas las palabras del diccionario que tienen ese sufijo, es decir, que terminan con el string **suffix**.

Se debe mostrar el siguiente output:

output	
1	Sufijo {suffix}:
2	palabra_1
3	.
4	.
5	.
6	palabra_k

Importante: La operación debe tener complejidad no mayor a $O(k \times L)$, donde k es la cantidad de palabras encontradas y L el largo de la palabra más larga encontrada.

Consideraciones generales

- Las palabras estarán compuestas por caracteres alfabéticos, sólo considerando minúsculas y sin incluir la ñ.
- El orden en que debes imprimir las palabras encontradas debe considerar el valor **ascii** de los caracteres. Esto implica seguir un orden alfabético.
- En los eventos FIND-PREFIX y FIND-SUFFIX, si no hay coincidencias, debes printear sólo la primera línea del output.
- Se recomienda investigar sobre la estructura de datos **Trie**.

Ejecución

Tu programa se debe compilar con el comando `make` y debe generar un ejecutable de nombre `dictionary` que se ejecuta con el siguiente comando:

```
./dictionary input output
```

donde `input` será un archivo con los eventos a simular y `output` el archivo donde se guardarán los resultados. En caso de querer correr el programa para ver los leaks de memoria utilizando `valgrind`, deberás utilizar el siguiente comando:

```
valgrind ./dictionary input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de `input` y `output`.

Input

El archivo de `input` comenzará con el entero `E` que indica el número de eventos por recibir. Luego, recibirás los `E` eventos correspondientes.

input	
1	7
2	WRITE 4
3	candidato
4	minutos
5	candados
6	creativos
7	FIND-WORD minutos
8	FIND-WORD precision
9	WRITE 2
10	precision
11	predicado
12	FIND-PREFIX cand
13	FIND-WORD precision
14	FIND-SUFFIX os

Output

output	
1	Se han escrito 4 palabras nuevas en el diccionario.
2	Palabra minutos encontrada.
3	Palabra precision no encontrada.
4	Se han escrito 2 palabras nuevas en el diccionario.
5	Prefijo cand:
6	candado
7	candidato
8	Palabra precision encontrada.
9	Sufijo os:
10	candados
11	creativos
12	minutos

Parte 2: ¡Descifra a Gru!



Problema

Gru ha desarrollado un sistema de código sólo utilizando las letras A, B, C, D. Logras interceptar un gran mensaje con todos sus planes, pero no es tan simple de descifrar. Por suerte tienes una lista de palabras que podrían ayudarte a entenderlo mejor. Tu misión es encontrarlas e indicar dónde encontrarlas a tu jefe Vector, para que no te despida.

Recibirás el mensaje de Gru, que será un **string** de tamaño N y Q palabras de tamaño M . Deberás entregar las posiciones donde inicia cada palabra dentro del mensaje. Se recomienda utilizar **rolling hashing**, como el [Algoritmo Rabin-Karp](#).

Consideraciones Generales

- Puedes asumir que $N > M$.
- Los índices se consideran desde 0 en adelante.
- En caso de que la palabra no se encuentre en el mensaje, el índice correspondiente es -1.
- Debes resolver el problema en tiempo $O(N + Q \times M)$.

Ejecución

Tu programa se debe compilar con el comando **make** y debe generar un ejecutable de nombre **decipher** que se ejecuta con el siguiente comando:

```
./decipher input output
```

donde **input** será un archivo con los eventos a simular y **output** el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando **valgrind** deberás utilizar el siguiente comando:

```
valgrind ./decipher input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

Primero recibirás el entero N , que indica el largo del mensaje a recibir. Luego, recibirás un **string** que corresponde al mensaje de Gru. Después, recibirás el entero M que representa el largo de las palabras a buscar en el mensaje. Finalmente, recibirás el entero Q , seguido de Q líneas con cada palabra a buscar.

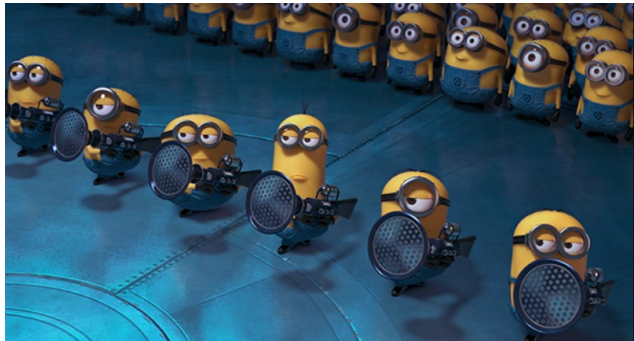
input	
1	30
2	BBBAAACCAABDDAACDABCAAADBDDBBAA
3	4
4	3
5	BBAA
6	DABC
7	BDAA

Output

Por cada consulta, deberás imprimir el número de consulta, seguido de todas las posiciones iniciales donde se encuentra la palabra en el mensaje:

output	
1	Consulta 0
2	1
3	26
4	Consulta 1
5	15
6	Consulta 2
7	-1

Parte 3: ¡Crea el arma definitiva!



Problema

La producción de dispositivos y chips ha terminado. Con la eficiente estructura de datos que diseñaste en la tarea pasada, ahora puedes obviar el tiempo de búsqueda y comenzar a resolver un nuevo problema... ¡Crear el arma definitiva! En esta parte deberás ensamblar pieza a pieza la nueva arma, preocupándote de que los dispositivos (ahora considerados piezas) calcen entre ellos sin incumplir ninguna restricción de incompatibilidad.

Entidades

Tablero: Plantilla del arma que consiste de una grilla con casillas de distintos tipos.

Casillas: Pueden ser libres “-”, de un color específico $\in \{A, C, V, M, R, N\}$, o estar bloqueadas “#”.

Piezas: Dispositivos que pueden ser asignados a casillas del tablero, siempre y cuando cumplan las restricciones especificadas. Tienen los atributos **WEIGHT**, **COLOR**, **UP**, **DOWN**, **LEFT** y **RIGHT**. Donde ‘weight’ es un número entero positivo, ‘color’ es el propio color de la pieza, y los demás atributos son los colores cada borde. Los colores se representan con un carácter $\in \{A, C, V, M, R, N\}$ ¹

Objetivo

En cada casilla no bloqueada del tablero puede colocarse exactamente una pieza. Solucionar el tablero consiste en, dado un conjunto de piezas disponibles, encontrar una forma de cubrir todas las casillas no bloqueadas utilizando esas piezas, de manera que se cumplan las restricciones descritas en la siguiente sección.

El objetivo es obtener el puntaje máximo al resolver el tablero. El puntaje se calcula como la suma de los pesos de todas las piezas utilizadas en la solución.

Por ejemplo, consideremos un tablero de 2x2 y 2 piezas, una con peso 4 y otra con peso 2. Si podemos solucionar el tablero usando 4 piezas de peso 2, entonces el puntaje será 8. Por otro lado, si usamos 4 piezas de peso 4, el puntaje será 16. Por lo tanto, el puntaje máximo del tablero es de 16.

Restricciones

1. Al colocar una nueva pieza en una casilla del tablero, los colores de sus bordes deben coincidir con los de las piezas adyacentes. Por ejemplo, si una pieza tiene el borde derecho (**RIGHT**) de color amarillo y se desea ubicar una pieza a su derecha, entonces el borde izquierdo (**LEFT**) de la nueva pieza también debe ser amarillo.
2. Si una pieza posee un borde de color negro (N), no se puede ubicar ninguna otra pieza en la casilla adyacente a dicho borde. (No se consideran como piezas las casillas bloqueadas “#” ni los bordes del tablero).
3. Si una casilla tiene un color específico, y se coloca en ella una pieza cuyo color coincide con el de la casilla, el valor de **WEIGHT** de dicha pieza se duplica al calcular el puntaje total.

¹Estos colores son: amarillo, celeste, verde, morado, rosado y negro respectivamente.

Consideraciones generales

- Puedes usar una cantidad ilimitada de cada una de las piezas entregadas.
- Las piezas no pueden ser rotadas.
- El color de la pieza y la casilla no necesariamente deben coincidir. Si coinciden sus colores, entonces se aplica el bonus ya explicado. Si no, la pieza se puede colocar normalmente, siempre y cuando cumpla con las otras restricciones.

Flujo

Al comienzo se entregará un entero Q que es la cantidad de consultas que recibirás. Luego, por cada consulta se te entregará lo siguiente:

- Dos enteros N y M , que representan las dimensiones del tablero $N \times M$. Seguido de N líneas con M caracteres que representan cada casilla del tablero.
- Un número K indicando el número de piezas distintas que se debes recibir. Seguido de K líneas describiendo las piezas en el formato **WEIGHT**, **COLOR**, **UP**, **DOWN**, **LEFT** y **RIGHT**, siendo **WEIGHT** un número entero positivo y el resto de atributos perteneciendo al conjunto $\{A, C, V, M, R, N\}$ de colores posibles.

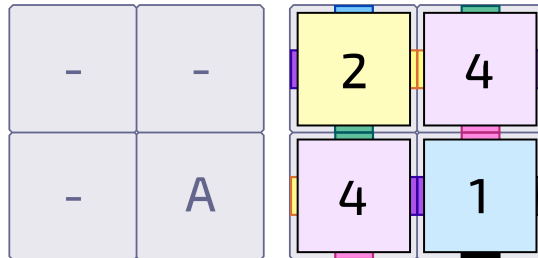
Finalmente, se espera como output el máximo puntaje obtenible al resolver el tablero, -1 es caso de que no se pueda resolver.

Ejemplo

input	
1	1
2	2 2
3	- -
4	- A
5	3
6	2 A C V M A
7	4 M V R A M
8	1 C R N M N

output	
1	11

En el ejemplo anterior, el output es 11, ya que el puntaje máximo que se puede alcanzar es usando dos piezas de peso 4, una de peso 2 y una de peso 1. Notar que el bonus no se activa, ya que la pieza colocada sobre la casilla de bonus no es de color amarillo. La siguiente imagen ilustra este tablero y la solución:



Ejecución

Tu programa se debe compilar con el comando **make** y debe generar un ejecutable de nombre **tiling** que se ejecuta con el siguiente comando:

```
./tiling input output
```

donde **input** será un archivo con los eventos a simular y **output** el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando **valgrind**, deberás utilizar el siguiente comando:

```
valgrind ./tiling input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Informe

Además del código de la tarea, se debe redactar un **breve** informe que explique cómo se pueden implementar algunas estructuras y algoritmos del enunciado. Les recomendamos **comenzar la tarea escribiendo este informe**, ya que pensar y armar un esquema antes de pasar al código ayuda a estructurarse al momento de programar y adelantar posibles errores que su solución pueda tener.

Este informe se debe entregar en un archivo PDF escrito usando LaTeX junto a la tarea (por lo cual tienen la misma fecha de entrega), con nombre `informe.pdf` en la raíz del repositorio. En este se debe explicar al menos los siguientes puntos:

- Parte 1
 1. Explica las estructuras y/o algoritmos utilizados para resolver cada evento. Menciona y justifica su complejidad algorítmica.
- Parte 2
 1. Explica la estrategia utilizada, justificando por qué resulta adecuada para este problema. Detalla cómo se manejaron las colisiones y cuál es el costo en memoria asociado a la aplicación de esta estrategia.
- Parte 3
 1. Explica la técnica de programación que usaste para resolver el problema. Incluye variables, dominios y restricciones en tu respuesta.
 2. Detalle al menos dos optimizaciones que aplicaste o podrían aplicarse a la técnica empleada.
- Bibliografía: Citar código de fuentes externas.

IMPORTANTE: Para asegurar una mejor comprensión y evaluación del informe, es **esencial** que cites el código al que haces referencia. Cada vez que menciones un fragmento de código, incluye el nombre del archivo y el número de línea correspondiente y su *hipervínculo* correspondiente. Puedes aprender cómo crear un *permalink* a tu código consultando la [documentación de GitHub](#). Esto facilitará la revisión y garantizará que el informe sea coherente con la implementación.

Ejemplo:

Si en la Parte 1 del informe estás explicando una función que implementa una determinada operación, la referencia al código podría ser así:

En nuestra implementación, utilizamos una lista enlazada para manejar la estructura de datos, como se muestra en la función `insertar_elemento` ([src/estructuras.c](#), línea 45). Esta elección permite...

De esta manera, aseguras que los revisores puedan localizar rápidamente el fragmento de código relevante y verificar que la explicación sea consistente con la implementación.

Nota: Si por alguna razón no lograste completar alguna parte del código mencionado en el informe, **aún puedes obtener puntaje** en la sección correspondiente. Para ello, debes describir claramente cuál era tu idea de implementación, los pasos que planeabas seguir, y explicar por qué no pudiste completarlo. Esto demuestra tu comprensión del problema y del proceso, lo cual también es valioso para la evaluación.

Finalmente, **puedes** referenciar código visto en clases sin necesidad de incluir el código o pseudocódigo en el informe.

Puedes encontrar un [template en Overleaf](#) disponible en este enlace para su uso en la tarea.

No se aceptarán informes de más de tres páginas (sin incluir bibliografía), informes ilegibles, o generados con inteligencia artificial.

Evaluación

La nota de tu tarea es calculada a partir de testcases de input/output, así como un informe escrito en LaTeX que explique un modelamiento sobre cómo abarcar el problema, las estructuras de datos a utilizar, etc.

La ponderación se descompone de la siguiente forma:

Informe (20 %)	Nota entre partes (70 %)	Manejo de memoria (10 %)
7 % Parte 1	20 % Tests Parte 1	5 % Sin leaks de memoria
6 % Parte 2	25 % Tests Parte 2	5 % Sin errores de memoria
7 % Parte 3	25 % Tests Parte 3	

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 1 segundo** y utilizar menos de 1 GB de RAM². De lo contrario, recibirás 0 puntos en ese test.

Recomendación de tus ayudantes

Estas tareas generalmente requieren de mucha dedicación, por lo que desde ya te recomendamos distribuir tu tiempo considerando los plazos definidos. Así mismo, te recomendamos fuertemente que antes de empezar a programar tu tarea, **leas el enunciado detenidamente y con anticipación para que te dediques a entender de manera profunda lo que te pedimos**. Una vez hayas comprendido el enunciado, dedica el tiempo que sea necesario para la planificación, modelación y elaboración de tu informe, para posteriormente poder programar de manera más eficiente. No olvides compilar el código como se indica en la tarea y de preguntar cualquier duda o por problemas que te surjan en [Discussions](#). Estos son consejos de tus ayudantes que te pueden ayudar a pasar el ramo.

Entrega

Código: GIT - Rama principal del repositorio asignado, que debes generar con el link dado al principio de este enunciado. Se entrega a más tardar el día de entrega a las 23:59 hora de Chile continental.

Atraso: Esta tarea considera la política de atraso y cupones especificada en el programa del curso.

Integridad académica

Este curso se adscribe al Código de Honor establecido por la Escuela de Ingeniería. Todo trabajo evaluado en este curso debe ser hecho **individualmente** por el alumno y **sin apoyo de terceros**. Se espera que los alumnos mantengan altos estándares de honestidad académica, acorde al Código de Honor de la Universidad. Cualquier acto deshonesto o fraude académico está prohibido; los alumnos que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada alumno conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Pregrado de la Escuela de Ingeniería.

Uso de IA

Se prohíbe el uso de herramientas de inteligencia artificial para la realización de esta tarea. Esto incluye, pero no se limita a, el uso de modelos de lenguaje como ChatGPT, Copilot, u otras tecnologías similares para la generación automática de código, soluciones, o cualquier parte del trabajo requerido. Nos reservamos el derecho de revisar todas las tareas entregadas con el fin de detectar el uso de inteligencia artificial en la creación de las soluciones. Las tareas en las que se determine que se ha utilizado IA serán consideradas como una infracción a la política de honestidad académica y serán tratadas como casos de copia.

²Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`